

Week 13: File Systems

TABLE OF CONTENTS

Complete Table of Contents

- 1 [Where are we?](#)
- 2 [Last Week: Intro to File Systems](#)
- 3 [Intro to file systems](#)
 - a [2. Files](#)
 - a [Possible Access Patterns](#)
 - b [A. contiguous allocation](#)
 - c [B. linked files](#)
 - d [C. indexed files](#)
 - b [3. Directories](#)
 - a [Approach 0: Users remember where on disk their files are](#)
 - b [Approach 1: Single directory for entire system](#)
 - c [Approach 2: Single directory for each user](#)
 - d [Approach 3: Hierarchical name spaces.](#)
 - e [Hierarchial Unix](#)
 - f [Links](#)
 - c [4. CS5600 File System](#)
 - a [A. fs5600 disk](#)
 - b [B. important data structures](#)
 - c [C. interface](#)
 - d [1. Last time](#)
 - e [2. fs5600 interface:](#)
 - f [3. Crash recovery](#)
 - a [A. Ad-hoc](#)
 - b [B. Copy-on-write approaches](#)
- 4 [More about Persistence Technologies](#)
 - a [Common Numbers and Performance](#)
 - b [How Driver Interfaces to Disk](#)
 - c [Technology and Systems Trends](#)
- 5 [SSD: Solid State Drives](#)

Agenda

- 1 Where are we?
- 2 File Systems, cont.
- 3 Lab 5 questions/comments/etc.

• Read()

• write()

• delete()

- Persist data

- Naming

- Directories

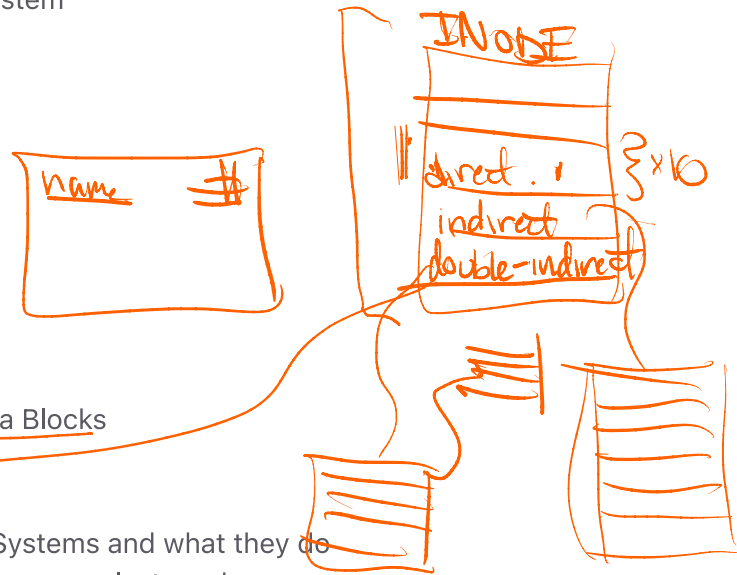
Where are we?

- Next week: Security!
- 2 weeks from tonight: Review
- 3 weeks from tonight: Final Exam!

Last Week: Intro to File Systems

We looked at:

- The Lab 5/Unix-like File System
- What happens when:
 - Open files
 - Create new files
 - Delete files
- Concepts:
 - Disk blocks
 - Inodes
 - Data Bitmap
 - Inode Bitmap
 - Multi-level Index to Data Blocks



Today:

- The bigger picture of File Systems and what they do
- Different options for things we saw last week
- Tradeoffs
- What things can go wrong and how to avoid big problems

Also:

- More about persistence: how the underlying drive/persistence technology impacts FS choices
 - Specifically, how SSD drives work (as compared to HDDs)

Intro to file systems

- What does a FS do?
 - 1 provide persistence
 - 2 give a way to "name" a set of bytes on the disk (files)
 - 3 give a way to map and organize human-friendly names to "names" (directories)
- a few quick notes about disks in the context of FS design
 - disk/SSD are the first thing we've seen that (a) doesn't go away; and (b) we can modify (BIOS ROM, hardware configuration, etc. don't go away, but we weren't able to modify these things).
 - two implications here:

- (i) we're going to have to put all of our important state on the disk
- (ii) we have to live with what we put on the disk!
 - scribble randomly on memory → reboot and hope it doesn't happen again.
 - scribble randomly on the disk → now what? (answer: in many cases, we're hosed.)
- where are FSES implemented?
 - can implement them on disk, over network, in memory, in NVRAM (non-volatile RAM), on tape, with paper (!!!!)

2. Files

- what is a file?
 - answer from user's view: a bunch of named bytes on the disk
 - answer from FS's view: collection of disk blocks
- big job of a FS: map name and offset to disk blocks

FS
 {file,offset} → disk address

[also called file mapping. see an introduction in S2 of this paper: (which btw is a cool paper about persistent memory) Neal, Ian, Gefei Zuo, Eric Shiple, Tanvir Ahmed Khan, Youngjin Kwon, Simon Peter, and Baris Kasikci. "Rethinking File Mapping for Persistent Memory.", FAST'21]

- operations are create(file), delete(file), read(), write()

**note that we have seen translation/indirection before:

page table:

page table
 virtual address → physical address

per-file metadata:

inode
 offset → disk block address

how'd we get the inode?

directory
 file name → inode

Analogies:

- per-process memory ↔ File
- multiple processes ↔ Multiple files
- page table ↔ inode
- memory chip ↔ disk/SSD

FS design parameters:

- File size:
 - small files (most files are small)
 - large files (much of the disk is allocated to large files)
- Access patterns:
 - sequential access vs.
 - random accesses vs.
 - keyed accesses
- disk utilization (metadata overhead and fragmentation)

Possible Access Patterns

Sequential:

- File data processed in sequential order
- By far the most common mode
- Example: editor writes out new file, compiler reads in file, etc

Random access:

- Address any block in file directly without linear scanning
- Examples: large data set, demand paging, databases

Keyed access

- Search for block with particular values
- Examples: associative database, index
- This thing is everywhere in the field of databases, search engines, but....
- ...usually not provided by a FS in OS

Possible Data Structures for FS Designs: A. contiguous B. linked files C. indexed files

A. contiguous allocation

"extent based":

- when creating a file, make user pre-specify its length, and allocate the space at once
- file metadata contains location and size
- example: IBM OS/360

```
[<free> a1 a2 a3 <5 free> b1 b2 <free> ]
```

BUT: what if a file c needs 7 sectors?!

- +: simple
- +: fast access, both sequential and random
- -: fragmentation

[somewhat regain interests due to PM: Li, Ruibin, Xiang Ren, Xu Zhao, Siwei He, Michael Stumm, and Ding Yuan. "ctFS: Replacing file indexing with hardware memory translation through contiguous file allocation for persistent memory." FAST'22]

B. linked files

"linked list based"

- keep a linked list of free blocks
- metadata: pointer to file's first block
- each block holds pointer to next one
- +: no more fragmentation
- +: sequential access easy (and probably mostly fast, assuming decent free space management, since the pointers will point close by)
- -: random access is a disaster
- -: pointers take up room in blocks; (overhead: $\text{sizeof}(\text{pointer}) / \text{sizeof}(\text{block})$); messes up alignment of data

C. indexed files

- Each file has an array holding all of its block pointers
 - like a page table, so similar issues crop up
- Allocate this array on file creation
- Allocate blocks on demand (using free list or bitmap)
- +: sequential and random access are both easy
- -: need to somehow store the array
- large possible file size → lots of unused entries in the block array
- large actual block size → huge contiguous disk chunk needed
 - solve the problem the same way we did for page tables.
- okay, so now we're not wasting disk blocks, but what's the problem?
 - (answer: equivalent issues as for page table walking: here, it's extra disk accesses to look up the blocks)
- this motivates the classic Unix file system
 - Unix inode:

```

permissions
times for file access, file modification, and inode-change
link count (# directories containing file)
ptr 1  --> data block
ptr 2  --> data block
ptr 3  --> data block
.....
ptr 11 --> indirect block
           ptr -->
           ptr -->
           ptr -->
           ptr -->
           ptr -->
ptr 12 --> indirect block
ptr 13 --> double indirect block
ptr 14 --> triple indirect block

```

This is just a tree.

- Question: why is this tree intentionally imbalanced?
 - (Answer: optimize for short files. each level of this tree requires a disk seek...)

Pluses/minuses:

- +: Simple, easy to build, fast access to small files
- +: Maximum file length can be enormous, with multiple levels of indirection
- -: worst case # of accesses pretty bad
- -: worst case overhead (such as 11 block file) pretty bad
- -: Because you allocate blocks by taking them off unordered freelist, metadata and data get strewn across disk
- Notes about inodes:
 - stored in a fixed-size array
 - Size of array fixed when disk is initialized; can't be changed
 - [why? easier for fs to find inodes, and fewer disk accesses (better performance!)]
 - Multiple inodes in a disk block (which is not the case for your Lab5)
 - Question: how many files can the following toy-fs has?
 - $\text{sizeof}(\text{inode}) = 128\text{B}$
 - $\text{sizeof}(\text{block}) = 512\text{B}$
 - toy-fs uses 1000 blocks to store inodes
 - [answer: the number of inodes (hence files) is $(512\text{B} / 128\text{B}) * 1000 = 4000$]
 - use `$ df -i ~` to see how many inodes you can use.
 - inode lives in known location, originally at one side of disk, now lives in pieces across disk (helps keep metadata close to data)
 - The index of an inode in the inode array is called an ***i-number***
 - Internally, the OS refers to files by ***i-number***
 - When a file is opened, the inode brought in memory
 - Written back when modified and file closed or time elapses

3. Directories

- Problem: "Spend all day generating data, come back the next morning, want to use it." F. Corbato, on why files/dirs invented.

[skipped, dir history]

Approach 0: Users remember where on disk their files are

- like remembering your social security or bank account #
 - yuck. (people want human-friendly names.)
- So use directories to map names to file blocks, somehow
- But what is in directory?
- A short history of directories

Approach 1: Single directory for entire system

- Put directory at known location on disk
- Directory contains `<name, inumber>` pairs
- If one user uses a name, no one else can
- "Many ancient personal computers work this way" [I heard this as an anecdote; never saw one myself...]

Approach 2: Single directory for each user

- Still clumsy, and "ls" on 10,000 files is a real pain
- (But some oldtimers still work this way)

Approach 3: Hierarchical name spaces.

- Allow directory to map names to files **or other dirs**
- File system forms a tree (or graph, if links allowed)
- Large name spaces tend to be hierarchical
- examples: IP addresses, domain names, scoping in programming languages, etc.
- more generally, the concept of hierarchy is everywhere in computer systems]

Hierarchical Unix

- used since CTSS (1960s), and Unix picked it up and used it nicely
 - structure like: [draw: "/" bin/ dev/ tmp/ usr/ ls, grep ...]
- directories stored on disk just like regular files
 - here's the data in a directory file; this data can be in the *data blocks* of the directory or else in the inode of the directory.

```
[<name, inode#>]
<bin, 1021>
<dev, 1001>
<sbin, 2011>
....
```

- i-node for directory contains a special flag bit
- only special users can write directory files
- key point: i-number might reference another directory
 - this neatly turns the FS into a hierarchical tree, with almost no work
- bootstrapping: where do you start looking?
 - root dir always inode #2 (0 and 1 reserved)
 - and, voila, we have a namespace!
- special names: "/", ".", ".."
- given those names, we need only two operations to navigate the entire name space:
 - "cd name": (change context to directory "name")
 - "ls": (list all names in current directory)
- example:

```
/a/foo.c
/b/c/essay.txt
```

What does the file system look like?

```
[ i0 ... i7 || [block ] [ block ] [block ] .....]
```

Links

- hard link: multiple dir entries point to same inode; inode contains refcount
 - `$ ln /tmp/a /tmp/b` creates a synonym (`b`) for file (`a`)
 - how do we avoid cycles in the graph? (answer: can't hard link to directories).
- soft link: synonym for a *name*
 - `$ln -s /tmp/a /tmp/sb`
 - creates a new inode (sb), not just a new directory entry
 - new inode has "sym link" bit set
 - contents of that new file: `/tmp/a`
- Question: what will happen: `$ rm /tmp/a; cat /tmp/b; cat /tmp/sb` ?
 - Try it yourself!
- Question: can I create soft-link cycles?
 - Yes, you can.
 - Try:

```
$ mkdir /tmp/a; mkdir /tmp/b
$ ln -s /tmp/a /tmp/b/a
$ ln -s /tmp/b /tmp/a/b
```

4. CS5600 File System

[this is an (almost) duplication of the Lab4 instructions.]

- a Unix-like fs
 - with many simplifications
 - for example, dirs are no deeper than 10

A. fs5600 disk

- an abstract disk (the overall system format)
 - a block is 4KB

```
+-----+-----+-----+-----+
| super | block | root dir | data blocks ... |
| block | bitmap | inode | |
+-----+-----+-----+-----+
block    0      1      2      3 ...
```

- talk to the disk?
 - read or write one or multiple blocks

[see handout week12.b, panel 1]

B. important data structures

- superblock

```
/* Superblock - holds file system parameters.
*/
struct fs_super {
```

```
uint32_t magic;
uint32_t disk_size; /* in blocks */

/* pad out to an entire block */
char pad[FS_BLOCK_SIZE - 2 * sizeof(uint32_t)];
};
```

** block bitmap

- tells fs5600 which blocks are free
- one bit represents one block
- QUESTION: what's the maximum size of a fs5600? [answer: 128MB = 4096 * 8 * 4KB]

** file and inode

```
/* inode = 4096 bytes */
struct fs_inode {
    uint16_t uid;
    uint16_t gid;
    uint32_t mode;
    uint32_t ctime;
    uint32_t mtime;
    int32_t size;
    uint32_t ptrs[INODE_NUM_PTRS];
};
```

- metadata [or stat, see full file metadata:
<https://pubs.opengroup.org/onlinepubs/7908799/xsh/sysstat.h.html>]

```
* uid, gid: user and group of this file
* mode (see below)
* ctime: changed time, time of last status (metadata) change
* mtime: modified time, time of last data modification
(fs5600 skips the access time in Unix)
* size: file size in bytes
```

- ptrs

[draw on board]

- Question: what's the maximum size of a file? [answer: number of ptrs = (4KB - 20B) / 4B = 1019 file size = 1019 * 4KB = 4076KB ≈ 4MB]
- mode (uint32_t) [see handout]

```
|<-- S_IFMT ---->|          |<-- user ->|<-- group ->|<-- world ->|
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| F | D |   |   |   |   | R | W | X | R | W | X | R | W | X |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

* R: read
```

```

* W: write
* X: execute

* F bit: 0100000 (the 16th bit from right)
* D bit: 0040000 (the 15th bit from right)

```

- directory
 - recall that directory is used to map name to #inode

```

/* Entry in a directory
*/
struct fs_dirent {
    uint32_t valid : 1;
    uint32_t inode : 31;
    char name[28]; /* with trailing NUL */
};

```

- Question: If a dir inode can use all the data blocks, how many files can appear in one directory?
 - one `fs_dirent` is 32B (see above)
 - A file is maximum of 4076KB.
 - So there can be $127 * 1024$ files, or roughly 128K ($\approx 4\text{MB}/32\text{B}$)

C. interface

1. Last time

- files * Unix inode [a pretty cool recent work: ctFS, where they reuse VM for fs. Li, R., Ren, X., Zhao, X., He, S., Stumm, M. and Yuan, D. ctFS: Replacing file indexing with hardware memory translation through contiguous file allocation for persistent memory. FAST'22] * fs5600 inode
- directories
 - special file with <name, inode number> mapping
 - links: hard links & soft links

Question: if I run `$ touch /tmp/a; ln /tmp/a /tmp/b; ln -s /tmp/a /tmp/sb; rm /tmp/a`

What is the output of this cmd ? `$ cat /tmp/b` What is the output of this cmd? `$ cat /tmp/sb` and why? [see note from last time]

- link cycles Question: can I create link cycles?
 - Yes. Question: can I create hardlink only cycles?
 - No. (by stop creating hardlink to dirs) Question: what can go wrong with hardlink cycles?
- fs5600 design
 - layout, superblock, bitmap, root dir, inode, dir entry, mode, ...
 - see Lab4 instructions

2. fs5600 interface:

- path walk

```
int inum = path2inum(char *pat);
```

- how? for example, `"/a/b/c/file"` [answer:
 - 1 split path to tokens: `["a", "b", "c", "file"]`
 - 2 starting from root dir (block 2)
 - 3 find `"a"` in `"/"` (block 2)
 - 4 get `"a"`'s inode
 - 5 get `"b"` in `"/a/"`
 - 6 ...]
- **fs_read** - read data from a file
 - how? for example, `"read("/a/file", buf, len, offset)"` (pseudocode) [answer:
 - 1 path walk to find the file's inode
 - 2 find offset's block (all data blocks compose a linear space)
 - 3 read len bytes]
- **fs_write** - write to a file
 - how? for example, `"write("/a/file", buf, len, offset)"` (pseudocode) [answer:
 - 1 path walk to find the file's inode
 - 2 allocate blocks if needed (`len+offset > size`)
 - 3 find offset's block
 - 4 write len bytes to the file]

Question: can you think of any metadata to update? [answer:

- mtime
- size]

- **fs_create** - create a new (empty) file
 - how? for example, `"create("/a/file", mode)"` (pseudocode) [answer: 1. path walk to get the inode of the parent folder (`"/a/"`) 2. allocate a block as `"file"`'s inode 3. add `"file"` to `"/a/"`]
- Question: how many `block_write()` do you think will happen in a `fs_create`? [answer: at least 4 times:
 - 1 to bitmap (for allocating new blocks)
 - 1 to parent dir inode (metadata update, mtime)
 - 1 to parent dir data block (adding `"file"`)
 - 1 to the `"file"`'s inode
 - and likely 1 data block to file's inode]
- **mkdir("/dir1/", 0644)**
 - Question: what does `"0644"` mean?
 - [answer: `"rw-r--"`: owner can RW; group can R; others can R]

[draw fs5600 layout and inode for `"/"`]

- how it works?
 - 1 path walk to get the inode of the parent folder (`"/"`)
 - 2 allocate an inode for `"dir1"` (say block#10)

- 3 allocate a block for data and init the block (say block#11)
- 4 add direntry "dir1" to parent dir's data (say block#3)
- 5 update parent inode (for example, mtime)

- Question: how many `block_write()` do you think will happen in this process?

[answer: 5 times:

- block#1: bitmap, for allocating new blocks
- block#10: create "dir1" inode
- block#11: init data block
- block#3: add direntry "dir1" to the parent dir ("/")
- block#2: update metadata of the parent dir]
- in fact these writes can happen in different order
 - depends on your implementation
 - OS buffer-cache
 - underlying storage (the hardware)

3. Crash recovery

- intro
- ad-hoc
- copy-on-write
- journaling
- There are a lot of data structures used to implement the file system: bitmap of free blocks, directories, inodes, indirect blocks, data blocks, etc.
 - We want these data structures to be *consistent*: we want invariants to hold
 - We also want to ensure that data on the disk remains consistent.
 - Thorny issue: *crashes* or power failures.
- Making the problem worse is:
 - (a) write-back caching and
 - (b) non-ordered disk writes.
- (a) means the OS delays writing back modified disk blocks.
- (b) means that the modified disk blocks can go to the disk in an unspecified order.
- Example: the above ``mkdir("/dir1", 0644)`

There are five writes:

- 1 block#1: bitmap, for allocating new blocks
- 2 block#10: create "dir1" inode
- 3 block#11: init data block
- 4 block#3: add direntry "dir1" to the parent dir ("/")
- 5 block#2: update metadata of the parent dir

Note: writing to one block is guaranteed to be atomic by hardware.

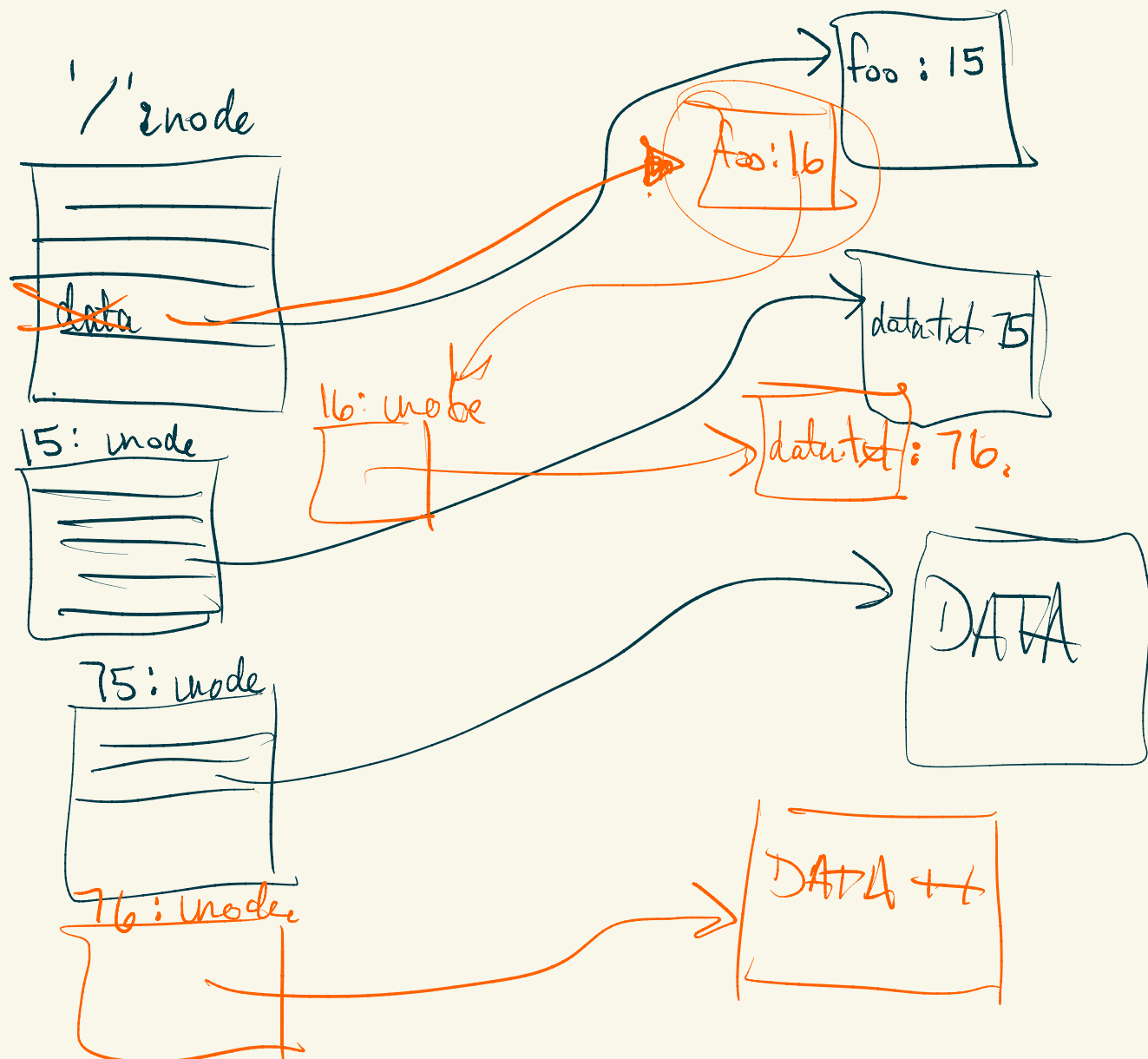
- crash.
- restart.

Ad-Hoc

Copy-on-Write

Write-Ahead Logging / Journaling

write /foo/data.txt



- uh-oh.
- Question: assume synchronous writes, what the consequences of crash when happening in-between:
 - 1 and 2? [losing track of two blocks]
 - 2 and 3? [unreachable "dir1", garbage in "dir1"]
 - 3 and 4? [unreachable "dir1"]
 - 4 and 5? [inconsistent metadata in "/"]
- Solution: the system requires a notion of atomicity
 - How to think about this stuff: imagine that a crash can happen at any time. (The only thing that happens truly atomically is a write of one or a few 512-byte disk sector.) So you want to arrange for the world to look sane, regardless of where a crash happens.
 - A challenge here is that metadata and data is spread across several disk blocks (and hence several sectors), so increasing size of atomic unit is not sufficient.
 - Your leverage, as file system designer, is that you can arrange for some disk writes to happen *synchronously* (meaning that the system won't do anything until these disk writes complete), and you can impose some ordering on the actual writes to the disk.
 - So we need to arrange for higher-level operations ("add data to file") to *look* atomic: an update either occurs or it doesn't.
 - Potentially useful analogy: during our concurrency unit, we had to worry about arbitrary interleavings (which we then tamed with concurrency primitives). Here, we have to worry that a crash can happen at any time (and we will tame this with abstractions like transactions). The response in both cases is a notion of atomicity.
- We will mention three approaches to crash recovery in file systems:
 - A. Ad-hoc (OSTEP calls this "fsck")
 - B. copy-on-write approaches
 - C. Journaling (also known as write-ahead logging)

A. Ad-hoc

- **Goal:** metadata consistency, not data consistency (rationale: too expensive to provide data consistency; cannot live without metadata consistency.)
- **Approach:** arrange to send file system updates to the disk in such a way that, if there is a crash, **fsck** can clean up inconsistencies
- example: `mkdir("/dir1", 0644)`

 - 1 block#1: bitmap, for allocating new blocks
 - 2 block#10: create "dir1" inode
 - 3 block#11: init data block
 - 4 block#3: add direntry "dir1" to the parent dir ("/")
 - 5 block#2: update metadata of the parent dir
- here are the fixes for this case:
 - 1 and 2? => recycle blocks
 - 2 and 3? => re-init dir1 [dangerous when there is seemingly correct info a fix: checksum]
 - 3 and 4? => send to "/lost+found/"
 - 4 and 5? => ? [likely ignore...]
- some other example cases:

- inode not marked allocated in bitmap → only writes were to unallocated, unreachable blocks; the result is that the write “disappears”
- inode allocated, data blocks not marked allocated in bitmap → fsck must update bitmap
- Disadvantages to this ad-hoc approach:
 - (a) fsck’s guarantees are unclear (hence ad-hoc)
 - (b) need to get ad-hoc reasoning exactly right (sometimes based on fs implementations)
 - (c) poor performance (synchronous writes of metadata)
 - multiple updates to same block require that they be issued separately. for example, imagine two updates to same directory block. requires first complete before doing the second (otherwise, not synchronous)
 - more generally, cost of crash recoverability is enormous. (a job like “untar” could be 10-20x slower)
 - (d) slow recovery: fsck must scan entire disk
 - recovery gets slower as disks get bigger. if fsck takes one minute, what happens when disk gets 10 times bigger?
 - essentially, fsck has to scan the entire disk

B. Copy-on-write approaches

- Goal: provide both metadata and data consistency, by using more space. Rationale: disks have gotten larger, space is not at a premium.
- Used by filesystems like ZFS, btrfs and APFS. [For more details read The Zettabyte File System by Jeff Bonwick, Matt Ahrens, Val Henson, Mark Maybee and Mark Shellenbaum. <https://www.cs.hmc.edu/~rhodes/courses/cs134/sp19/readings/zfs.pdf>]
- Approach: never modify a block, instead always make a new copy. In detail:

The filesystem has a root block, which we refer to as the Uberblock (copying terminology from ZFS). The uberblock is the **only** block in the filesystem that is ever *modified* (as opposed to being fully written, which the rest of the blocks are).

- An abstract example: update a leaf block
 - [draw a tree with checksum]
 - remember: *never modify, only copy*. so the file system allocates a new block, and writes the new version of the data to the new block
 - but that in turn necessitates writing a new version of the inode (to point to the new version of the block)
 - and that in turn *changes the inode number*, which means that parents and any directories hard-linking to the file have to change (for this to work, the inode has to store the list of hard links.)
 - and that in turn means that *those* directories’ inodes have to change
 - and so on up to the uberblock.
 - the change is *committed* – in the crucial sense that after a crash the new version will be visible – when and only when the uberblock is modified on disk.

More about Persistence Technologies

- 1 last time
 - I/O

- disk continued

Common Numbers and Performance

C. Common #s and performance

- capacity: high 100s of GB, now TBs common
- platters: 8
- number of cylinders: tens of thousands or more
- sectors per track: ~1000
- RPM: 10000
- transfer rate: 50-150 MB/s

Question: can you guess how long is disk's "mean time between failures"?

- mean time between failures: ~1 million hours (~100years) (for disks in data centers, it's vastly less; for a provider like Google, even if they had very reliable disks, they'd still need an automated way to handle failures because failures would be common (imagine 10 million disks: *some* will be on the fritz at any given moment). so what they do is to buy defective and cheap disks, which are far cheaper. lets them save on hardware costs. they get away with it because they *anyway* needed software and systems – replication and other fault-tolerance schemes – to handle failures.)

How Driver Interfaces to Disk

D. How driver interfaces to disk

- Sectors [see again handout's bootloader code from last time]
 - Disk interface presents linear array of **sectors**
 - traditionally 512 bytes (moving to 4KB)
- disk maps logical sector # to physical sectors
 - Zoning: puts more sectors on longer tracks
 - Track skewing: sector 0 position varies by track, but let the disk worry about it. Why? (for speed when doing sequential access)
 - Sparing: flawed sectors remapped elsewhere
- all of this is invisible to OS. stated more precisely, the OS does not know the logical to physical sector mapping.
- in old days (before 1990ish): the OS specifies a platter, track, sector (CHS, Cylinder-head-sector); but who knows where it really is?
 - nowadays, the OS sees a disk as an array of sectors (LBA, logical block addressing); normally each sector is of size 512B.
- Question: how many bits do we need to address a 1TB disk? (note: we will simplify here, assuming $1\text{TB}=10^{40}\text{B}$; in reality, in the context of storage, $1\text{TB}=1000,000,000,000\text{B}$, or 1 trillion bytes)
 - Answer:
 - 1 sector is $512\text{B} = 2^9$ Bytes;
 - the entire disk has $1\text{TB}/512\text{B} = 2^{40} / 2^9 = 2^{31}$ sectors;
 - to address each sector, we need at least 31-bits
 - In fact: "The current 48-bit LBA scheme was introduced in 2003 with the ATA-6 standard,[4] raising the addressing limit to $2^{48} \times 512$ bytes, which is exactly 128 PiB or approximately 144 PB." (from wiki: https://en.wikipedia.org/wiki/Logical_block_addressing)]

Technology and Systems Trends

E. technology and systems trends

- unfortunately, while seeks and rotational delay are getting a little faster, they have not kept up with the huge growth elsewhere in computers.
- transfer bandwidth has grown about 10x per decade
- the thing that is growing fast is disk density (byte_stored/\$). that's because density is less about the mechanical limitations
- to improve density, need to get the head close to the surface.
 - [aside: what happens if the head contacts the surface? called "head crash": scrapes off the magnetic material ... and, with it, the data.]
- Disk accesses a huge system bottleneck and getting worse. So what to do?
 - Bandwidth increase lets system (pre-)fetch large chunks for about the same cost as small chunk.
 - So trade latency for bandwidth if you can get lots of related stuff at roughly the same time. How to do that?
 - By clustering the related stuff together on the disk. can grab huge chunks of data without incurring a big cost since we already paid for the seek + rotation.

In fact, local network latency is much smaller than disk now.

[Diego Ongaro, Stephen M. Rumble, Ryan Stutsman, John Ousterhout, and Mendel Rosenblum. Fast Crash Recovery in RAMCloud. SOSP'11]

- The saving grace for big systems is that memory size is increasing faster than typical workload size
 - result: more and more of workload fits in file cache, which in turn means that the profile of traffic to the disk has changed: now mostly writes and new data.
 - which means logging and journaling become viable (more on this over next few classes)

SSD: Solid State Drives

1 SSD: solid state drives

[see handout week11.a]

- hardware organization
 - semiconductor-based flash memory
 - storing data electrically, instead of magnetically
 - a flash bank contains blocks
 - blocks (or erase blocks) are of size 128KB or 256KB
 - a block contains pages
 - pages are of 4KB to 16KB
- operations
 - read: a page
 - erase: a block, resetting all bits to 1

- program: a page, setting some bits to 0
 - (you cannot program a page twice without erasing)
- (logical) write: a combination of erase and program operations
- **Question:** can you imagine how to update a page A in a single block flash? (which of course is a little bit too small...) [answer: 1. copy other pages in the block to other places (where? anywhere, memory or disk) 2. erase the entire block 3. write page A with wanted contents 4. copy other pages back to their positions]
 - this echos “writes are more expensive than reads” which appears in many places. (probably something deeper about it.)
- performance
 - read: tens of us
 - erase: several ms
 - program: hundreds of us
- a bummer: wear-out
 - a block can bare about 10,000 to 100,000 times erasing, then becomes unusable
- FTL: flash translation layer
 - read/write logic blocks →FTL→ read/erase/program physical blocks (note, the “blocks” in logical blocks and physical blocks are different: logical blocks as in the device interface, physical blocks as in the flash hardware)
 - Question: if you were FTL, how would you mitigate wear-outs? [answer: evenly spread the erase/program to blocks.]
 - a log-structured FTL
 - idea:
 - when write, appending the write to the next free page (called logging).
 - when read, keeping track where the data are by mapping logical data to physical pages.

**** an example:**

- Given a flash bank has three blocks; each has two pages.
- there are four writes to pages: `wirte(logic_page_1)` [short as LP1] `wirte(logic_page_10)` [short as LP10] `wirte(logic_page_99)` [short as LP99]
- what will happen:

+-----+ blocks	block 0	block 1	block 2
+-----+ pages	P1	P2	P3 P4 P5 P6
+-----+ data	LP1	LP10 LP99	
+-----+			

mapping: LP1 => P1, LP10 => P2, LP99 => P3

Question: what will happen if the following op is write(logic_page_1')? [answer:

+-----+	blocks		block 0		block 1		block 2	
+--+--+---	pages		P1		P2		P3 P4	
			P5		P6			

```

+-----+-----+-----+-----+-----+ data |LP1 |LP10|LP99|LP1'| | |
+-----+-----+-----+-----+-----+

```

mapping: LP1 => P4, LP10 => P2, LP99 => P3

]

- Notice: P1 now contains old (invalid) data and is useless.
- hence, SSD requires *garbage collection* (GC)
 - want to GC block0
 - move the useful pages (i.e., P2[LP10]) in the same block to other free places
 - erase block 0 (now bot P1 and P2 can be used again)
- complicated internals, hence sometimes unpredictable latency
 - predicting latency? see below

[Hao, Mingzhe, Levent Toksoz, Nanqinqin Li, Edward Edberg Halim, Henry Hoffmann, and Haryadi S. Gunawi. "LinnOS: Predictability on unpredictable flash storage with a light neural network.", OSDI'20]

This site uses [Just the Docs](#), a documentation theme for Jekyll.